

LYCHEEMEMORY V2: Efficient Long-Term Memory for LLM Agents via Semantic Segment-Level Consolidation

Dongfang Li^{1*}, Zixuan Liu^{1*}, Junmai Wang¹, Jiahe Huang¹,
Fuhao Li¹, Bonian Jia¹, Baotian Hu^{1†}, Min Zhang¹
¹Harbin Institute of Technology, Shenzhen

Abstract

Long-horizon LLM agents must preserve information from past interactions to support future tasks. Existing memory systems typically rely on eager consolidation, invoking LLMs after each interaction to extract, summarize, or update memories. This design makes memory construction increasingly costly as conversations grow. Coarse summarization can reduce construction cost but risks discarding fine-grained contextual evidence, whereas larger retrieval contexts or multi-hop LLM reasoning shift the overhead to query time. We present LYCHEEMEMORY V2, an efficient long-term memory framework that replaces turn-level consolidation with semantic segment-level consolidation. Instead of consolidating every interaction, LYCHEEMEMORY batches multiple exchanges into segments and encodes each finalized segment into context-independent typed memory records. Segment-level batching lowers LLM encoding frequency, while semantic boundary detection helps preserve coherent event-level and temporal evidence compared with fixed-window batching. The resulting records are organized with lightweight structured indexes for query-planned evidence retrieval. Experiments using GPT-4.1-Mini show that LYCHEEMEMORY achieves state-of-the-art performance, reaching 89.22% on LoCoMo and 92.20% on LongMemEval-S. Compared with A-Mem, it reduces construction tokens by 86.0% on LoCoMo and 75.9% on LongMemEval-S without increasing query-time token usage. More broadly, our results suggest that the accuracy-cost trade-off of long-term agent memory depends not only on what information is retained, but also on the granularity at which it is consolidated.

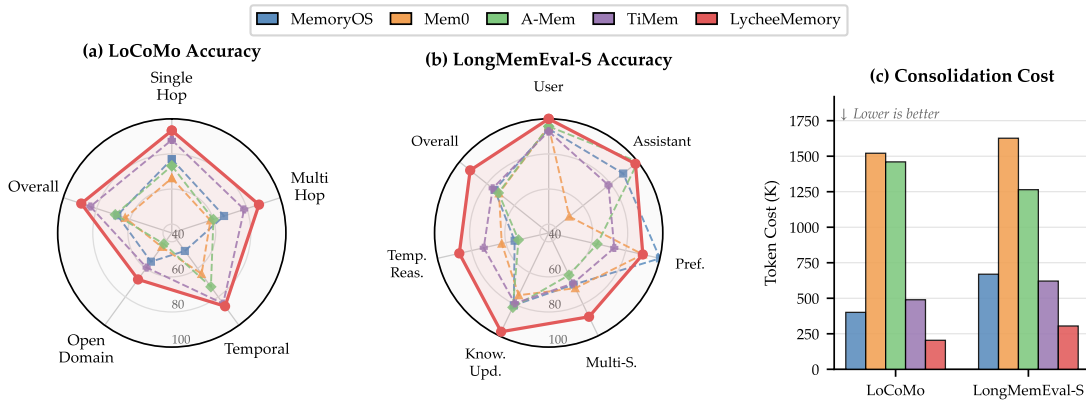


Figure 1: LYCHEEMEMORY achieves superior accuracy with substantially lower consolidation cost under GPT-4.1-Mini. **Left:** Category-wise accuracy comparison on LoCoMo, covering single-hop, multi-hop, temporal, open-domain, and overall questions. **Middle:** Category-wise accuracy comparison on LongMemEval-S, including user, assistant, preference, multi-session, knowledge-update, temporal-reasoning, and overall questions. **Right:** Memory consolidation cost comparison across LoCoMo and LongMemEval-S, where lower is better.

* Equal contribution.

†Corresponding Author

Contents

1	Introduction	4
2	Related Work	5
2.1	Long-Term Memory for LLM Agents	5
2.2	Memory Construction for LLM Agents	5
2.3	Memory Organization and Retrieval	6
2.4	Efficient Memory Systems	6
3	Method	7
3.1	Overview	7
3.2	Online Semantic Segmentation	7
3.3	Segment-Level Memory Encoding	9
3.4	Structured Evidence Organization	9
3.5	Plan-Guided Multi-Route Retrieval	10
4	Experiments	10
4.1	Experimental Setup	10
4.2	Main Results	13
4.3	Accuracy-Cost Trade-off	13
4.4	Ablation Study	14
5	Conclusion	15
A	Experimental Protocol Details	19
A.1	Benchmark Data and Evaluation Tasks	19
A.1.1	LoCoMo	19
A.1.2	LongMemEval-S	19
A.2	Answer Generation and Judge Protocol	20
A.3	Metrics and Token Accounting	21
B	Algorithmic Description	21
B.1	Semantic Segment-Level Memory Construction	22
B.2	Plan-Guided Multi-Route Retrieval	22
C	Implementation Details	22
C.1	Online Semantic Segmentation	22
C.2	Segment-Level Memory Encoding	24
C.3	Structured Evidence Organization	24
C.4	Plan-Guided Multi-Route Retrieval	24
D	Token Accounting Details	25

D.1 Construction Tokens	26
D.2 Query Tokens	26
E Ablation Variant Definitions	26
E.1 Construction-Side Variants	26
E.2 Representation-Side Variants	26
E.3 Retrieval-Side Variants	27
F Prompt Templates and Evaluation Prompts	27
F.1 LYCHEEMEMORY Segment Encoding Prompt	27
F.2 LYCHEEMEMORY Query Planning Prompt	28
F.3 LYCHEEMEMORY Answer Generation Prompt	30
F.4 LongMemEval-S Official Judge Prompts	32
F.5 LoCoMo Accuracy Metric and Judge Prompt	33
G Limitations and Artifact Use	33

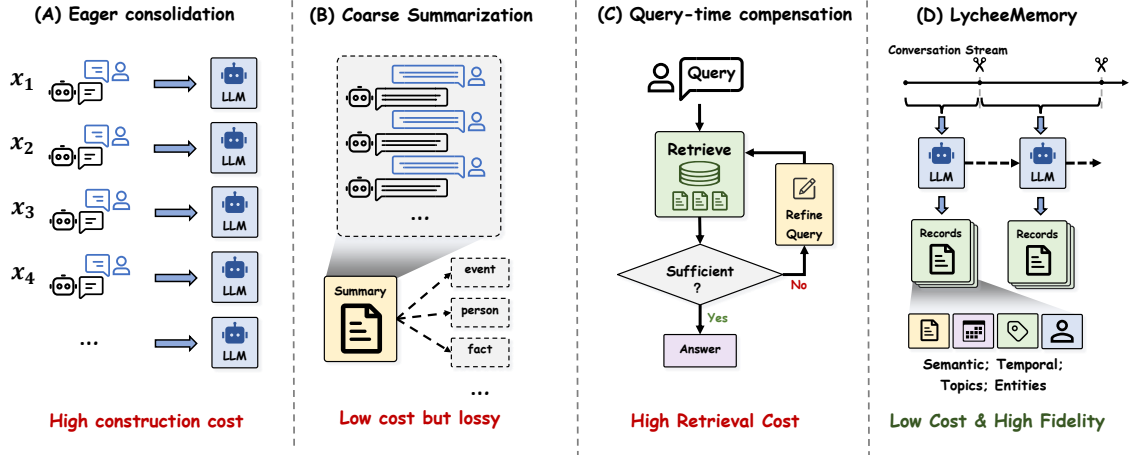


Figure 2: Motivation of LYCHEEMEMORY. (A) Eager turn-level consolidation incurs high construction cost. (B) Coarse summarization reduces construction cost but may discard fine-grained evidence. (C) Iterative query-time compensation shifts overhead to retrieval. (D) LYCHEEMEMORY batches multiple exchanges into semantically coherent segments and encodes each finalized segment into typed evidence records for structured retrieval, reducing construction frequency while preserving fine-grained evidence without query-time expansion.

1 Introduction

Large language model (LLM) agents (Zhong et al., 2024; Packer et al., 2023; Memobase, Inc., 2025) are increasingly expected to operate as persistent assistants rather than single-turn responders. In long-horizon settings (Maharana et al., 2024; Wu et al., 2025) such as personal assistance, customer support, tutoring, and task-oriented dialogue, an agent (Zhong et al., 2024; Chhikara et al., 2025) must remember user preferences, historical events, evolving facts, constraints, and previous failures across many interactions.

Since an LLM cannot reliably keep an unbounded interaction history in its active context window, recent systems (Packer et al., 2023; Kang et al., 2025; Chhikara et al., 2025; Xu et al., 2025; Choi et al., 2026; Hsu et al., 2026) have introduced external memory modules to extract, update, organize, and retrieve information from past conversations. A common design in these systems (Chhikara et al., 2025; Xu et al., 2025; Kang et al., 2025; Hu et al., 2026; Kim et al., 2026) is eager consolidation: after each turn or short exchange, an LLM is invoked to summarize the interaction, extract facts, update existing memories, or build links with historical memories. Although effective, this turns memory construction into a high-frequency LLM operation (Hu et al., 2026; Choi et al., 2026; Kim et al., 2026; Stabile & Zimuel, 2026) whose token cost accumulates rapidly as conversations grow.

Simply making memory cheaper by storing coarser summaries is also insufficient, because long-term memory questions (Maharana et al., 2024; Wu et al., 2025) often depend on fine-grained evidence (Choi et al., 2026; Hsu et al., 2026; Li et al., 2026d) such as entities, temporal expressions, coreference relations, and small contextual details. Meanwhile, improving recall (Li et al., 2026c) by expanding top- k retrieval or using LLM-driven multi-hop search (Li et al., 2026d; Chen et al., 2026a; Li et al., 2026a) introduces additional query-time overhead. Thus, the key challenge is not merely how to store more history, but how to preserve sufficient evidence at the right granularity while keeping both construction and retrieval costs under control.

Motivated by these limitations, we propose LYCHEEMEMORY V2, a cost-efficient long-term memory framework for LLM agents (Figure 2). The core idea is that past experience should re-enter future reasoning with the proper granularity, temporal relation, and evidence strength. Instead of consolidating every turn, LYCHEEMEMORY batches multiple exchanges and invokes the LLM once for each finalized segment, thereby reducing construction frequency. Within this segment-level batching scheme, semantic surprise and cohesion signals determine where segments end, helping preserve coherent event boundaries rather than relying on mechanical fixed windows. Each segment

is then encoded into context-independent typed memory records that contain natural-language statements, memory types, entities, topics, temporal scopes, and links to the original dialogue evidence. To maintain continuity across segments without re-reading the full history, the system carries lightweight disambiguation feedback, including entity aliases and reference relations, into later consolidation steps. LYCHEEMEMORY further organizes records in an append-only structured memory store with entity, topic, temporal, event-frame, and entity-topic indexes. At query time, a planner decomposes the user question into multiple recall routes and retrieves evidence from semantic memory, structured indexes, temporal indexes, and raw turns, followed by route-level reranking, fusion, and diversity-aware selection. This design improves evidence coverage without relying on blind context expansion.

We evaluate LYCHEEMEMORY on LoCoMo and LongMemEval-S (Maharana et al., 2024; Wu et al., 2025). The results show that LYCHEEMEMORY consistently improves long-term memory QA accuracy while substantially reducing construction-token cost, achieving over 20-point accuracy gains over A-Mem (Xu et al., 2025) with up to 7.2× fewer construction tokens. These improvements are obtained with lower query-time token usage than A-Mem, with particularly large gains on multi-hop, temporal-reasoning, preference-tracking, and multi-session questions.

Our contributions are threefold:

- We identify memory-construction granularity as an important efficiency lever for long-term agent memory: batching multiple exchanges reduces write-side LLM calls, while semantic boundaries preserve coherent evidence relative to fixed windows.
- We introduce LYCHEEMEMORY, a semantic segment-level memory construction framework that encodes each finalized segment into typed, self-contained records. Bounded cross-segment context maintains entity and reference continuity despite less frequent consolidation.
- We demonstrate a stronger accuracy–cost trade-off on LoCoMo and LongMemEval-S, improving long-term memory QA accuracy while reducing construction-token cost and query-time token usage relative to A-Mem. Ablations further distinguish the cost effect of segment-level batching from the accuracy effect of semantic boundary selection.

2 Related Work

2.1 Long-Term Memory for LLM Agents

Long-term memory has become a central component of LLM agents that must operate across sessions rather than within a single context window. Early systems (Zhong et al., 2024; Packer et al., 2023; Memobase, Inc., 2025) introduced external memory to preserve user facts, preferences, and historical context, thereby moving from stateless response generation toward persistent assistance. More recent frameworks broaden this agenda by separating different forms of memory or by coupling memory with agent control. For example, MemoryOS (Kang et al., 2025) and A-Mem (Xu et al., 2025) emphasize richer memory abstractions and organization, while G-Long (Choi et al., 2026) and HORMA (Hsu et al., 2026) connect memory with more structured reasoning or navigation mechanisms. In parallel, benchmarks such as LoCoMo (Maharana et al., 2024) and LongMemEval (Wu et al., 2025) clarify that long-term memory quality depends not only on whether information is retained, but also on whether it can be reconstructed and used under long-horizon budget constraints.

Taken together, this literature suggests that long-term memory systems are increasingly differentiated along three technical axes: how memories are constructed from interaction streams, how they are organized and retrieved for downstream reasoning, and how cost is controlled during long-horizon use. We organize the remainder of this section around these three axes.

2.2 Memory Construction for LLM Agents

A dominant line of work constructs memory eagerly from incoming dialogue. Mem0 (Chhikara et al., 2025) extracts and updates memories at turn granularity, while A-Mem (Xu et al., 2025) enriches this pattern with linked notes and more structured memory units. MemoryOS (Kang et al., 2025)

and related frameworks (Nan et al., 2025; Wang, 2026) introduce stronger memory hierarchies or management policies, but still rely on LLM-mediated ingestion or summarization as new dialogue arrives. The common strength of this family is semantic richness at write time; its main drawback is that memory construction remains a frequent LLM operation.

A second line of work reduces write-side cost after or around this basic pipeline. MemRefine (Kim et al., 2026) treats compression as a post-construction decision problem, using the LLM to delete, merge, or preserve entries under a budget. DMF (Stabile & Zimuel, 2026) removes LLM calls from memory management through deterministic scoring and decay, trading semantic flexibility for lower and more reproducible cost. MemRouter (Hu et al., 2026) makes a different trade-off by learning an embedding-based admission policy that decides whether a new interaction should enter memory. These methods show that construction can be made cheaper in different ways, but most of them either optimize memory after frequent writes have already been produced or make storage decisions at relatively fine temporal granularity.

Recent systems have also explored segment-level memory construction. SeCom (Pan et al., 2025) partitions conversations into topically coherent segments and compresses them before retrieval, while HiMem (Zhang et al., 2026) uses topic-aware event-surprise segmentation to construct episode memories within a hierarchical memory architecture.

LYCHEEMEMORY shares this segment-level construction granularity but focuses specifically on online write-side efficiency. It uses embedding-based boundary decisions as a construction trigger, allowing multiple exchanges to share one encoding call instead of producing separate turn-level updates. Semantic boundary detection determines the composition of each batch, while bounded cross-segment disambiguation supports the construction of self-contained records.

2.3 Memory Organization and Retrieval

Once memory has been written, the next challenge is how to retrieve the right evidence for complex reasoning. Many early systems (Zhong et al., 2024; Memobase, Inc., 2025; Chhikara et al., 2025) expose memory primarily through flat semantic retrieval over summaries, notes, or profile-like records. This design works well for direct factual lookup, but it is less well aligned with questions that depend on temporal constraints, multi-hop relations, or comparisons across episodes (Maharana et al., 2024; Wu et al., 2025), where entity identity, topical scope, and temporal validity must be handled explicitly.

A substantial body of work addresses this problem by adding structure to the memory store itself. G-Long (Choi et al., 2026) and REAL (Lu et al., 2026) organize memory through structured relations or graph-like representations, while hierarchical or workspace-style systems such as HORMA (Hsu et al., 2026), MAGE (Chen et al., 2026b), and Infini-Memory (Ji et al., 2026) support reasoning across multiple granularities. These methods show that richer structure can improve long-horizon retrieval, especially when evidence must be composed rather than directly matched. At the same time, the amount of extra indexing, maintenance, or traversal machinery varies substantially across systems, so the trade-off is better characterized as additional structural complexity rather than a uniform cost increase.

Another line of work improves retrieval through more explicit query-time control. EviMem (Li et al., 2026d) retrieves evidence iteratively based on missing information, MemFlow (Chen et al., 2026a) routes a query to different memory operations, and MemReranker (Li et al., 2026a) shows that reasoning-aware reranking can substantially improve final evidence quality. These approaches make clear that long-term memory retrieval is often operation-aware rather than single-shot. LYCHEEMEMORY is closest in spirit to this structured and routed line, but it emphasizes a lighter query-time design: it uses one planning step to dispatch recall over pre-built indices instead of relying on iterative retrieval loops.

2.4 Efficient Memory Systems

Recent memory research increasingly treats efficiency as a primary objective rather than a secondary implementation concern. EMBER (Li et al., 2026c) studies budgeted evidence retention, asking which information should remain available under a fixed memory budget. ActiveMem (Jiang et al., 2026) reduces the burden on the main model by offloading memory distillation to lighter components,

while LightMem (Fang et al., 2025) develops a lightweight memory-augmented generation pipeline. ScrapMem (Chang & Ren, 2026) focuses on storage-constrained multimodal personalization, while Memanto (Abtahi et al., 2026) argues that some graph-plus-vector memory designs impose avoidable infrastructure and latency overheads. DMF (Stabile & Zimuel, 2026) and MemRouter (Hu et al., 2026) similarly illustrate that large efficiency gains are possible when parts of memory management are replaced by deterministic or lightweight policies.

This literature establishes an important evaluation principle: memory systems should be compared not only by answer quality but also by their token, latency, and storage footprints. LYCHEEMEMORY belongs to this efficiency-oriented line, but its emphasis is narrower and more specific. Its main focus is construction-time token cost, and its key distinction is to reduce that cost by lowering the frequency of semantic consolidation rather than primarily by compressing stored memory or simplifying retrieval infrastructure.

3 Method

3.1 Overview

We formulate long-term agent memory as an online construction and retrieval problem. Given a multi-session conversation stream $C = \{x_1, \dots, x_T\}$, where each exchange x_t contains a user message and the corresponding assistant response, the system incrementally builds a memory store $\mathcal{M}$ and later retrieves evidence for a query q . The objective is to maximize answer accuracy while controlling both construction cost, measured by total LLM tokens used during memory building, and query cost, measured by LLM tokens consumed per question.

LYCHEEMEMORY replaces turn-level consolidation with semantic segment-level consolidation. Instead of invoking an LLM after every exchange, it buffers multiple exchanges and performs one encoding pass per finalized segment. This segment-level batching reduces construction frequency. A semantic boundary detector separately determines which exchanges belong to each segment so that the batching policy follows conversational structure rather than fixed windows. Each segment is converted into typed, context-independent memory records, enriched with entities, topics, temporal scopes, and provenance links. The records are stored in an append-only evidence store with semantic retrieval and structured indexes for entity, topic, temporal, event-frame, and entity-topic access. At query time, a planner decomposes the question into typed recall routes, and deterministic retrieval modules collect, fuse, and select evidence from the indexed memory store.

Figure 3 provides an overview of LYCHEEMEMORY, which consists of a construction phase and a retrieval phase. During construction, online semantic segmentation groups the conversation stream into coherent segments and triggers segment-level encoding only when a segment is finalized. The encoded records are then inserted into an append-only store and exposed through structured evidence indexes. During retrieval, the query planner converts the current question into typed recall routes, and the retrieved candidates are fused into a compact evidence context for answer generation. Section 3.2 describes online segmentation, Section 3.3 defines segment-level record construction, Section 3.4 presents structured evidence organization, and Section 3.5 describes plan-guided retrieval. Algorithmic descriptions are provided in Appendix B.

3.2 Online Semantic Segmentation

The first component decides when a dialogue fragment is sufficiently complete to be consolidated. Turn-level eager consolidation is costly because it applies the same LLM encoding operation to every exchange, including exchanges that are still part of an unfinished semantic episode. Batching multiple exchanges into one segment reduces this call frequency. Fixed windows provide such batching mechanically, but they ignore event boundaries and may split temporally or referentially coherent evidence. We therefore use online semantic segmentation as the boundary policy within segment-level batching, triggering consolidation when the active segment becomes semantically saturated or a topic transition is detected.

Let $S_k = \{x_s, \dots, x_{t-1}\}$ denote the current active segment before observing a new exchange x_t . We encode each exchange into an embedding vector $\mathbf{e}_t$, maintain the segment centroid $\mathbf{c}_k$, and keep the

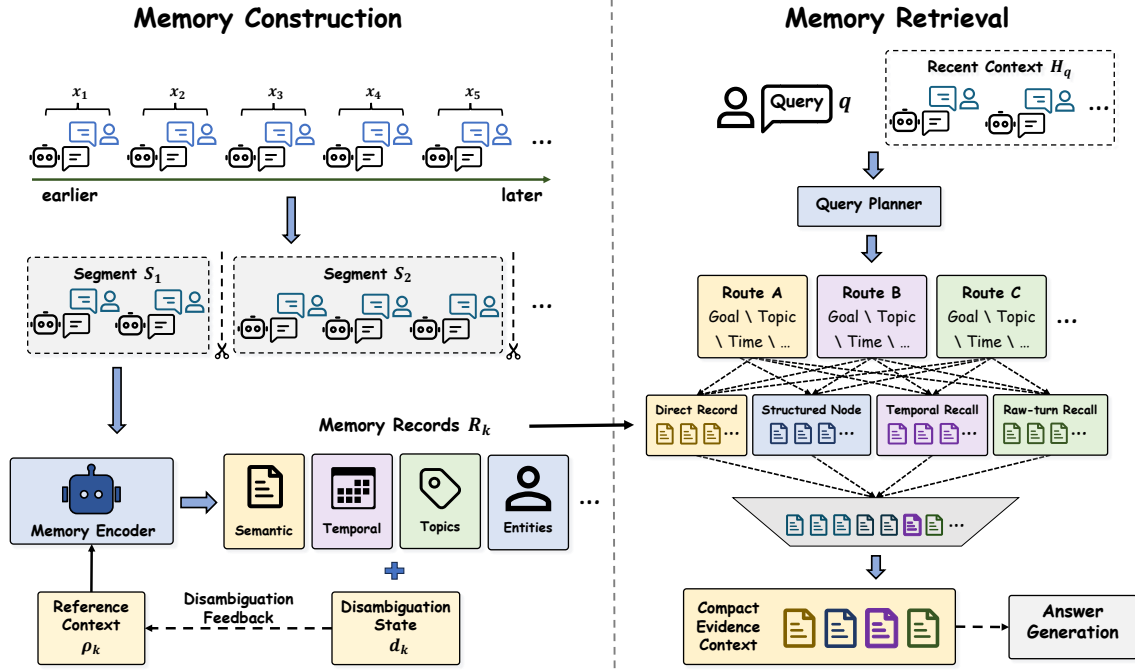


Figure 3: Overview of LYCHEEMEMORY. During memory construction, online semantic segmentation partitions the conversation stream into coherent segments, and the memory encoder converts each finalized segment into typed records with semantic, temporal, topic, and entity information. A compact reference context and disambiguation feedback maintain continuity across segments. During memory retrieval, the query planner converts the current query and recent context into route-specific evidence needs. Direct-record, structured-node, temporal, and raw-turn recall collect candidates, which are fused into a compact evidence context for answer generation.

most recent exchange embedding $\mathbf{h}_k$. The semantic surprise score s_t is defined as

$$s_t = 1 - \max(\text{sim}(\mathbf{e}_t, \mathbf{c}_k), \text{sim}(\mathbf{e}_t, \mathbf{h}_k)), \quad (1)$$

where $\text{sim}(\cdot, \cdot)$ is cosine similarity. This score captures whether the new exchange departs from both the global topic of the active segment and the local conversational trajectory.

We also measure how much the new exchange weakens internal segment coherence. For a set of exchange embeddings V , let

$$\text{Coh}(V) = \frac{1}{|V|} \sum_{\mathbf{v} \in V} \text{sim}(\mathbf{v}, \mathbf{c}_V), \quad (2)$$

where $\mathbf{c}_V$ is the centroid of V . The cohesion drop d_t induced by x_t is

$$d_t = \max(0, \text{Coh}(S_k) - \text{Coh}(S_k \cup \{x_t\})). \quad (3)$$

A larger drop indicates that adding the exchange would make the segment less topically coherent.

The final boundary score combines semantic surprise, cohesion drop, token pressure, and turn-count pressure:

$$p_t = \sigma(b + w_s \phi(s_t) + w_c d_t + w_l L_t + w_n N_t), \quad (4)$$

where $\phi(\cdot)$ applies local normalization to semantic surprise, L_t is the normalized token-length pressure, and N_t is the normalized turn-count pressure. A segment is finalized when p_t exceeds a fixed threshold δ or a hard token cap is reached; otherwise, x_t is appended to the active segment. This design keeps the boundary decision fully embedding-based, so mid-segment exchanges are buffered without LLM inference. The reduction in encoding calls comes from consolidating at segment rather than exchange granularity: the number of calls is proportional to the number of segments $|S|$ rather than the number of exchanges T . The semantic boundary score determines the composition of those segments.

3.3 Segment-Level Memory Encoding

The second component converts each finalized segment into memory records that can be retrieved and interpreted without the original dialogue context. Raw turns often contain pronouns, elliptical references, relative dates, and implicit entity mentions. If such turns are stored directly, later retrieval must recover missing context at query time. LYCHEEMEMORY instead resolves these dependencies at write time within a coherent segment, where the necessary local context is still available.

For a finalized segment S_k , the encoder receives the segment text together with a compact reference context ρ_k from recent segments. It produces a set of memory records $\mathcal{R}_k$ and an updated disambiguation state d_k :

$$(\mathcal{R}_k, d_k) = \text{Encode}(S_k, \rho_k). \quad (5)$$

The encoding prompt asks the LLM to perform three operations in a single pass: extract atomic information units, resolve coreference and elliptical mentions, and normalize relative temporal expressions using the session timestamp.

Each record $r_i \in \mathcal{R}_k$ is represented as

$$r_i = (\text{id}_i, \tau_i, \text{text}_i, \mathcal{E}_i, \mathcal{K}_i, \mathcal{T}_i, \text{src}_i), \quad (6)$$

where id_i is an internal record identifier, τ_i is the memory type, text_i is a self-contained natural-language statement, $\mathcal{E}_i$ is the entity set, $\mathcal{K}_i$ is the topic-tag set, $\mathcal{T}_i$ records normalized event or validity times when available, and src_i stores provenance links to the source turns. The memory type τ_i is selected from a finite schema covering facts, preferences, events, constraints, procedures, failure patterns, and tool affordances.

To preserve continuity across segment boundaries, the encoder also returns a disambiguation state d_k containing resolved aliases, canonical entity names, and reference relations that may be needed by later segments. The reference context for the next segment is constructed as

$$\rho_{k+1} = [d_k; \text{Recent}(\mathcal{R}_{k-m:k})], \quad (7)$$

where $\text{Recent}(\cdot)$ selects a bounded set of recent record summaries from the same session. This context is truncated to a fixed budget, which prevents the prompt size from growing with the full conversation history. The result is a sequence of records that are locally self-contained while remaining consistent across segments.

3.4 Structured Evidence Organization

The third component organizes encoded records so that retrieval can satisfy both semantic and structured constraints. Flat vector search is effective for approximate semantic matching, but long-term memory questions often require explicit access by entity, topic, time, event context, or combinations of these fields. LYCHEEMEMORY therefore builds structured evidence indexes directly from the metadata already produced by the segment encoder, without additional LLM calls.

For each record r_i , the organizer inserts the record into a vector store and updates a structured store. The vector store embeds text_i for direct semantic retrieval. The structured store maintains five classes of evidence nodes:

- entity nodes, one for each entity in $\mathcal{E}_i$;
- topic nodes, one for each topic tag in $\mathcal{K}_i$;
- entity-topic nodes, one for each co-occurring pair $(e, k) \in \mathcal{E}_i \times \mathcal{K}_i$;
- temporal nodes, keyed by day-level or month-level temporal scopes in $\mathcal{T}_i$;
- event-frame nodes, each grouping the records produced from the same finalized semantic segment while retaining the source session and turn range as provenance.

Each evidence node stores a searchable text representation, an optional embedding, and pointers to linked memory records.

The structured store retains textually distinct statements as separate records. Direct record search retrieves records by semantic similarity, while evidence-node search retrieves or filters structured nodes and expands them to linked records. Temporal nodes support exact-date and range-based

filtering, and event-frame nodes preserve local co-occurrence among records extracted from the same finalized segment. Evidence distributed across segments or sessions is assembled later by retrieving and fusing records from multiple evidence nodes and recall routes. Because all nodes are derived from record metadata, this phase requires only embedding and bookkeeping operations, and the write-side LLM cost remains dominated by segment-level encoding.

3.5 Plan-Guided Multi-Route Retrieval

The final component retrieves evidence for a query using one planning step followed by deterministic recall. Long-term memory questions are often heterogeneous: a temporal question may require date filtering, a multi-hop question may require evidence from multiple entities, and a preference question may require stable user facts rather than recent events. A single embedding query cannot express these distinct evidence requirements. Iterative LLM retrieval can address the issue, but it increases query-time token cost. LYCHEEMEMORY separates query understanding from evidence collection: the LLM is used once to specify recall routes, while the subsequent retrieval operations are non-generative.

Given a query q and recent dialogue context H_q , the planner outputs a structured plan

$$\Pi(q, H_q) = (y, \{R_1, \dots, R_m\}), \quad (8)$$

where y is the question type and each route $R_j = (g_j, Q_j, C_j, T_j)$ contains a route goal, one or more search queries, structured constraints, and optional temporal constraints. The question type selects retrieval parameters such as channel weights, per-channel candidate limits, and expansion depth.

Each route executes four recall channels in parallel. Direct record recall searches the record vector store using route-specific search queries. Evidence-node recall searches entity, topic, entity-topic, and event-frame nodes, then expands matched nodes to their linked records. Temporal recall applies exact date or range constraints over temporal nodes. Raw-turn recall searches verbatim dialogue embeddings to recover evidence that may not have been consolidated into typed records. Each candidate receives channel-specific scores and retains its provenance.

Candidates from all routes are merged with reciprocal-rank fusion:

$$\text{RRF}(d) = \sum_{j=1}^m \frac{1}{\kappa + \text{rank}_j(d)}, \quad (9)$$

where $\text{rank}_j(d)$ is the rank of candidate d under route R_j and κ is a smoothing constant. Candidates are optionally reranked by a lightweight cross-encoder within each route before route-level lists are fused. The fused candidates are then passed through diversity-aware selection so that evidence from different routes is preserved. The selected records and raw-turn snippets are serialized with their memory types, temporal scopes, and provenance metadata to form the final context for answer generation. When retrieved evidence conflicts, the answer model is instructed to prefer the most recent supported information.

This retrieval design provides structured evidence coverage while bounding generative token usage. The only generative query-time operation is the planning call. All subsequent recall, expansion, reranking, fusion, and selection steps are embedding lookups, structured filtering, or arithmetic scoring.

4 Experiments

4.1 Experimental Setup

Datasets. We evaluate LYCHEEMEMORY on two long-term conversational memory benchmarks that cover different interaction patterns. **LoCoMo** (Maharana et al., 2024) focuses on companion-style casual sharing scenarios, comprising 10 multi-session conversations (averaging ~600 turns, ~16K tokens) with 1,986 question-answer pairs in total. Following the standard long-term memory QA setting, we retain 1,540 questions from the single-hop, multi-hop, temporal-reasoning, and open-domain categories to test whether memory systems can recover fine-grained evidence from cross-session history. **LongMemEval-S** (Wu et al., 2025) focuses on agent-style task-oriented interactions

with significantly longer dialogue contexts (500 conversations, averaging $\sim 115\text{K}$ tokens), covering six categories: user facts, assistant facts, preference tracking, multi-session reasoning, knowledge update, and temporal reasoning, posing stricter challenges for memory construction and retrieval under long-term high-load scenarios. Together, these benchmarks cover the core scenario we address: dialogue history grows continuously, and systems must preserve usable evidence under limited construction and query budgets. Detailed benchmark composition is provided in Appendix A.1.

Baselines. We compare against baselines covering different design directions. **Full Context** places the entire dialogue history directly into the LLM context window, serving as a direct long-context reference without external memory. **Naive RAG** chunks the conversation and retrieves via embedding similarity, representing standard retrieval-augmented approaches. Among dedicated long-term memory systems, **Mem0** (Chhikara et al., 2025) invokes the LLM after each turn for fact extraction and update, representing the standard eager consolidation paradigm; **A-Mem** (Xu et al., 2025) extends this with Zettelkasten-style linked notes and dynamic memory evolution, representing an agentic memory design with linked and evolving records; **MemoryOS** (Kang et al., 2025) employs an OS-inspired layered memory architecture (short-term / mid-term / long-term persona modules), representing more complex memory organization directions. Additionally, we compare with **MemOS** (Li et al., 2025), **Nemori** (Nan et al., 2025), **LightMem** (Fang et al., 2025), and **TiMem** (Li et al., 2026b) on both benchmarks, and with **MemU** (NevaMind-AI, 2025) on LoCoMo.

Implementation. We evaluate LYCHEEMEMORY using GPT-4.1-Mini and GPT-4o-Mini as backbones. Each backbone is used throughout the pipeline for memory encoding, query planning, and final answer generation, with temperature set to 0. Embeddings are computed with `text-embedding-3-small`, and retrieval-side reranking uses `bge-reranker-v2-m3`. Additional implementation and prompt details are provided in Appendices C and F.

Answer and judge protocol. Within each backbone setting, all methods use the same benchmark split and judge protocol. For LYCHEEMEMORY, the selected backbone is also used for memory encoding and query planning. Outputs in both settings are evaluated by GPT-4o-Mini using benchmark-specific judge prompts. On LoCoMo, the judge receives the question, gold answer, and generated answer and accepts semantically equivalent phrasings; category accuracy is computed within each retained question type, and overall accuracy over all 1,540 retained questions. On LongMemEval-S, we use the official task-specific yes/no judge with a 10-token output cap. Preference questions use rubric-style desired responses. For knowledge-update questions, the judge accepts the updated answer even if previous information is also mentioned; for temporal-reasoning questions, it applies the official duration tolerance. Judge calls are excluded from all token counts.

Metrics. We report overall accuracy and fine-grained category accuracy following each benchmark’s official categorization. Overall accuracy is a micro-average over the valid evaluation set: 1,540 retained questions for LoCoMo and 500 questions for LongMemEval-S, with no additional filtering. Compared with lexical overlap metrics such as F1 and BLEU-1, the LLM judge more directly assesses semantic correctness in open-ended generation. Lexical overlap can underestimate semantically correct answers that differ in phrasing and overestimate answers that share surrounding words but contain an incorrect critical fact.

Token accounting. For the efficiency analysis, we use GPT-4.1-Mini and count construction and query tokens as the sum of generative LLM input and output tokens. For every method, construction tokens cover all generative memory-building calls, whereas query tokens cover final answer generation and any method-specific query-time generative operations. For LYCHEEMEMORY, construction tokens comprise the segment-encoding prompt, finalized segment text, reference/disambiguation context, memory-record output, and disambiguation-state output; query tokens comprise the query-planner and final answer-generation input/output, including serialized evidence and recent dialogue context. Embedding computation, deterministic indexing, retrieval and filtering, reciprocal-rank fusion, non-generative reranking, diversity-aware selection, and judge calls are excluded; final answer generation is excluded from construction but included in query cost. Construction cost is averaged per conversation or conversation-question instance, query cost per question, and both are reported in thousands (K). Reporting both quantities separates write-side from query-side cost and reveals whether gains merely shift computation between stages.

Table 1: Main results on LoCoMo with GPT-4.1-Mini and GPT-4o-Mini. Accuracy (%) is reported for the official question types and the overall score; higher is better. Best results for each model are bold, and LYCHEEMEMORY results are shown in blue.

Backbone	Method	Single Hop	Multi Hop	Temporal	Open Domain	Overall
GPT-4.1-Mini	Full Context	90.84	82.62	79.13	57.29	84.80
	Naive RAG	61.24	58.87	33.96	50.00	54.42
	MemOS	85.37	79.43	75.08	64.58	80.84
	MemU	74.91	72.34	43.61	54.17	66.62
	MemoryOS	77.05	66.31	47.66	55.21	67.60
	Mem0	66.23	58.16	63.86	44.79	62.92
	A-Mem	73.25	59.93	72.90	42.71	68.83
	Nemori	84.90	75.10	77.60	51.00	79.40
	LightMem	81.57	62.77	64.17	54.17	72.79
	TiMem	87.99	78.37	84.74	59.38	83.77
	LYCHEEMEMORY	93.34	87.23	86.60	67.71	89.22
GPT-4o-Mini	Full Context	90.01	69.86	52.96	55.21	76.43
	Naive RAG	57.55	54.61	24.30	51.04	49.68
	MemOS	81.45	69.15	72.27	60.42	75.97
	MemU	72.77	62.41	33.96	46.88	61.17
	MemoryOS	74.55	58.87	44.24	46.88	63.64
	Mem0	65.52	52.84	52.02	37.50	58.64
	A-Mem	66.83	52.13	61.99	30.21	60.84
	Nemori	82.10	65.30	71.00	44.80	74.40
	LightMem	76.61	67.02	76.32	45.83	72.87
	TiMem	81.43	62.20	77.63	52.08	75.30
	LYCHEEMEMORY	84.90	68.09	80.06	54.17	78.90

Table 2: Main results on LongMemEval-S with GPT-4.1-Mini and GPT-4o-Mini. Accuracy (%) is reported for each official question category and the overall score; higher is better. Best results for each model are bold, and LYCHEEMEMORY results are shown in blue.

Backbone	Method	SSU	SSA	SSP	MS	KU	TR	Overall
GPT-4.1-Mini	Full Context	91.43	100.00	56.67	53.38	75.64	48.12	66.20
	Naive RAG	85.71	82.36	83.33	63.91	75.64	45.86	67.20
	MemOS	90.00	64.29	50.00	54.14	70.51	63.91	65.20
	MemoryOS	94.29	89.29	100.00	67.67	80.77	54.89	74.40
	Mem0	95.71	50.00	90.00	69.92	74.36	62.41	71.20
	A-Mem	95.71	100.00	63.33	61.65	82.05	52.63	71.60
	Nemori	90.00	92.90	86.70	55.60	79.50	72.20	74.60
	LightMem	90.00	23.21	76.67	51.13	88.46	84.21	69.60
	TiMem	92.86	78.57	73.33	66.92	79.49	72.93	75.80
	LYCHEEMEMORY	100.00	98.21	90.00	87.97	97.44	87.22	92.20
GPT-4o-Mini	Full Context	44.29	80.36	53.33	35.34	55.13	34.59	45.60
	Naive RAG	85.71	83.93	46.67	52.63	65.38	41.35	59.40
	MemOS	95.71	67.86	96.67	70.67	74.26	77.44	77.80
	MemoryOS	97.14	89.29	70.00	58.65	73.08	48.87	67.80
	Mem0	95.71	55.36	70.00	57.89	69.23	54.14	64.40
	A-Mem	92.54	98.21	36.67	53.38	70.51	44.36	63.20
	Nemori	88.60	83.90	46.70	51.10	61.50	61.70	64.20
	LightMem	87.14	32.14	68.18	71.74	83.12	67.18	69.81
	TiMem	95.71	82.14	63.33	70.83	86.16	68.42	76.88
	LYCHEEMEMORY	97.14	96.43	70.00	72.93	74.36	72.18	78.80

4.2 Main Results

Tables 1 and 2 show that LYCHEEMEMORY achieves the highest overall accuracy on both benchmarks with either backbone. LYCHEEMEMORY reaches 89.22% on LoCoMo and 92.20% on LongMemEval-S with GPT-4.1-Mini, compared with 84.80% for Full Context and 75.80% for TiMem, the strongest baselines on the respective benchmarks. The corresponding results with GPT-4o-Mini are 78.90% and 78.80%, improving over the strongest baselines by 2.47 and 1.00 percentage points. Section 4.3 further compares the accuracy and token costs of MemoryOS, Mem0, A-Mem, TiMem, and LYCHEEMEMORY using GPT-4.1-Mini. The consistent gains across both benchmarks indicate that LYCHEEMEMORY remains effective with either backbone and across different interaction lengths.

LoCoMo. On LoCoMo, LYCHEEMEMORY’s largest gains over A-Mem with GPT-4.1-Mini appear in multi-hop reasoning (87.23% vs. 59.93%, +27.3 pp) and open-domain questions (67.71% vs. 42.71%, +25.0 pp). It also reaches 93.34% on single-hop questions, +20.1 pp above A-Mem (73.25%). On temporal reasoning, LYCHEEMEMORY reaches 86.60%, +13.7 pp above A-Mem (72.90%) and +38.9 pp above MemoryOS (47.66%). LYCHEEMEMORY achieves 78.90% overall with GPT-4o-Mini, 18.06 pp above A-Mem and 2.47 pp above Full Context, together with 80.06% on temporal reasoning. These gains show that LYCHEEMEMORY remains effective across direct recall, evidence composition, temporal reasoning, and open-ended questions; Section 4.4 further examines the corresponding design choices. Full Context remains competitive on LoCoMo, but with either backbone, its accuracy decreases on the substantially longer conversations in Table 2.

LongMemEval-S. On LongMemEval-S, LYCHEEMEMORY’s three largest gains over A-Mem with GPT-4.1-Mini occur in temporal reasoning (87.22% vs. 52.63%, +34.59 pp), preference tracking (90.00% vs. 63.33%, +26.67 pp), and multi-session reasoning (87.97% vs. 61.65%, +26.32 pp). It also reaches 97.44% on knowledge-update questions, compared with 82.05% for A-Mem, showing that the pipeline can recover updated evidence from the retained history under the benchmark’s answer-level criterion (Wu et al., 2025). On single-session user and assistant facts, LYCHEEMEMORY reaches 100.00% and 98.21%, respectively. LYCHEEMEMORY achieves the highest overall accuracy of 78.80% with GPT-4o-Mini, 1.00 pp above MemOS, together with the highest multi-session score (72.93%) and a tied best result on single-session user facts (97.14%). These results show consistent gains across temporal, preference-intensive, and cross-session evidence needs with both models. MemoryOS reaches 100.00% on preference tracking with GPT-4.1-Mini, higher than LYCHEEMEMORY’s 90.00%, suggesting that dedicated user-profile modules may provide advantages for preference-intensive queries.

4.3 Accuracy-Cost Trade-off

Using GPT-4.1-Mini, we next examine whether the accuracy gains require higher write- or query-side token consumption. Figure 4 visualizes the trade-off, while the discussion below reports the key exact values.

Construction cost. On LoCoMo, LYCHEEMEMORY uses only 204.1K construction tokens, 86.0% and 86.6% lower than A-Mem’s 1459.9K and Mem0’s 1520.8K respectively, and 58.3% lower than TiMem’s 489.5K. On LongMemEval-S, LYCHEEMEMORY uses 304.7K, 75.9% lower than A-Mem’s 1264.3K and 50.9% lower than TiMem’s 620.9K. Construction savings come from segment-level batching: it reduces LLM encoding calls from T (one per turn) to $|S|$ (one per segment, averaging 5.8 turns per segment on LoCoMo), while each call processes multiple exchanges rather than an individual turn. Semantic boundary detection determines which exchanges are batched together; its contribution relative to fixed-window batching is evaluated separately in Section 4.4.

Query cost. Despite substantial accuracy gains, LYCHEEMEMORY’s query tokens do not increase. On LoCoMo, LYCHEEMEMORY uses 4.01K query tokens, lower than A-Mem’s 5.56K (-27.9%) and TiMem’s 10.71K (-62.6%). On LongMemEval-S, it uses 8.88K, lower than A-Mem’s 15.46K (-42.6%) and TiMem’s 11.36K (-21.8%). This indicates that LYCHEEMEMORY’s accuracy gains do not come from expanding query-time context or adding multi-step LLM reasoning. Its retrieval pipeline requires only one LLM planning call; the remaining recall, scoring, fusion, and selection operations are embedding lookups and arithmetic computations that consume no generative tokens.

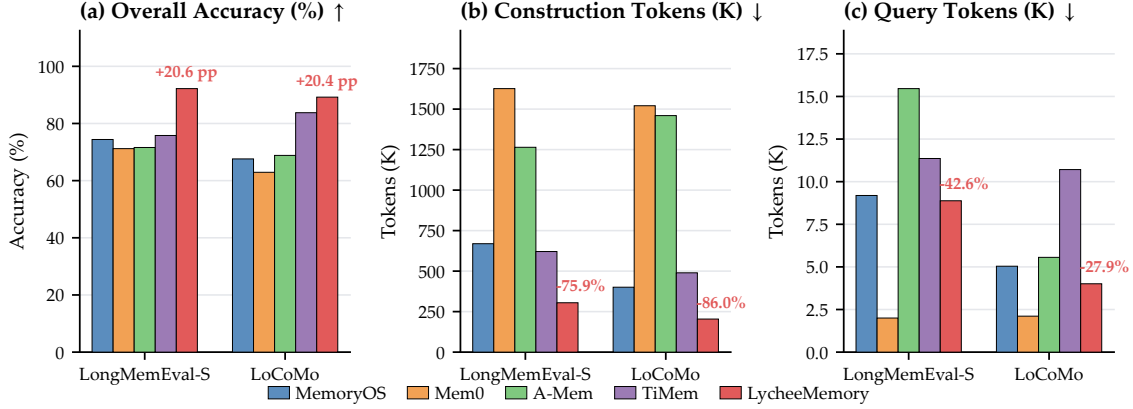


Figure 4: Accuracy-cost comparison using GPT-4.1-Mini. The panels report overall accuracy, construction tokens, and query tokens on LongMemEval-S and LoCoMo. LYCHEEMEMORY is highlighted in red; annotations show its changes relative to A-Mem.

Table 3: Construction-side ablation on LoCoMo with GPT-4.1-Mini. Eager construction removes segment-level batching, whereas fixed-window construction preserves batching but replaces semantic boundary detection with mechanical windows; higher accuracy and lower construction tokens are better.

Variant	Overall	Single	Multi	Temp.	Open	Const. ↓
Full LYCHEEMEMORY	89.22	93.34	87.23	86.60	67.71	204.1
eager cons.	81.88	87.16	74.47	81.31	59.38	849.9
fixed-window cons.	82.40	90.49	74.82	75.70	56.25	174.7

Overall trade-off. Taken together, LYCHEEMEMORY simultaneously improves accuracy and reduces both construction and query token consumption relative to A-Mem and TiMem. Mem0 has the lowest query tokens but also the lowest accuracy, while MemoryOS has relatively low construction cost but accuracy still falls far below LYCHEEMEMORY. Overall, LYCHEEMEMORY provides a stronger trade-off among accuracy, construction cost, and query-time overhead.

4.4 Ablation Study

Unless otherwise stated, all ablation experiments are conducted on LoCoMo using GPT-4.1-Mini. We assess the contributions of major design choices and component groups to the accuracy-cost trade-off following the system pipeline: construction triggering (Table 3), memory representation and cross-segment continuity (Table 4), and query-time retrieval (Table 5). We further vary the boundary threshold used by online semantic segmentation to assess parameter sensitivity (Figure 5). Detailed variant definitions are provided in Appendix E.

Construction triggering. Replacing segment-level batching with eager construction causes accuracy to drop from 89.22% to 81.88% (-7.3 pp) while construction tokens increase from 204.1K to 849.9K (+316%). Fixed-window consolidation retains batching and uses slightly fewer construction tokens than the full system (174.7K vs. 204.1K), but its accuracy drops to 82.40% (-6.8 pp), with the largest losses on multi-hop (87.23% → 74.82%) and open-domain questions (67.71% → 56.25%). Together, these comparisons show that batching reduces construction cost, while semantic boundary detection improves accuracy relative to mechanical windows.

Table 4: Representation-side ablation on LoCoMo with GPT-4.1-Mini. Summary-level records replace typed records, and the context variant removes cross-segment reference feedback; higher accuracy and lower construction tokens are better.

Variant	Overall	Single	Multi	Temp.	Open	Const. ↓
Full LYCHEEMEMORY	89.22	93.34	87.23	86.60	67.71	204.1
summary records	80.78	88.35	74.47	73.83	56.25	99.7
w/o cross-seg. context	81.56	87.87	75.18	76.95	60.42	189.6

Table 5: Retrieval-side ablation on LoCoMo with GPT-4.1-Mini. Record-vector only jointly removes structured and raw-turn recall, while w/o fusion/rerank/div jointly removes the selection modules. Higher accuracy and lower query tokens are better.

Variant	Overall	Single	Multi	Temp.	Open	Query ↓
Full LYCHEEMEMORY	89.22	93.34	87.23	86.60	67.71	4.01
record-vector only	81.75	88.23	76.60	78.19	52.08	4.09
w/o query planner	83.38	90.01	79.08	78.50	54.17	2.26
w/o fusion/rerank/div	66.62	71.34	60.64	66.04	44.79	3.35

Memory representation. Replacing typed, self-contained records with summary-level records reduces construction tokens to 99.7K, while accuracy drops to 80.78% (-8.4 pp) and temporal reasoning falls from 86.60% to 73.83% (-12.8 pp). This comparison shows higher accuracy for structured records than for summary-level memory. Removing cross-segment reference context leaves construction tokens nearly unchanged (189.6K vs. 204.1K) but reduces accuracy to 81.56% (-7.7 pp), showing the contribution of cross-segment continuity to the representation configuration.

Retrieval pipeline. Using record-vector retrieval only, query tokens remain comparable to the full system (4.09K vs. 4.01K) but accuracy drops to 81.75% (-7.5 pp), showing the combined effect of entity, topic, temporal, event-frame, and raw-turn recall. Removing the query planner reduces query tokens to 2.26K but accuracy drops to 83.38% (-5.8 pp), exposing a trade-off between the planning call and QA accuracy. Jointly disabling fusion, reranking, and diversity-aware selection reduces accuracy from 89.22% to 66.62% (-22.6 pp), showing the contribution of the combined evidence-selection stack.

Boundary-threshold sensitivity. To assess the stability of online semantic segmentation with respect to its cut criterion, we vary the threshold δ applied to the boundary probability p_t from 0.30 to 0.70 on the same LoCoMo evaluation set while keeping all other settings fixed. As shown in Figure 5, overall accuracy ranges from 88.18% to 89.22%, a total variation of 1.04 percentage points. The default threshold of 0.50 achieves the highest accuracy; the neighboring settings of 0.40 and 0.60 remain within 0.39 and 0.58 percentage points of the default, respectively. These results show limited sensitivity to the boundary threshold within the evaluated range.

Summary. Overall, removing or replacing the proposed construction, representation, and retrieval components produces substantial accuracy losses or cost increases, whereas accuracy remains stable across the evaluated values of δ . This contrast indicates that the observed gains arise from the system design rather than a narrowly tuned boundary threshold.

5 Conclusion

We introduced LYCHEEMEMORY, a cost-efficient long-term memory framework for LLM agents that replaces turn-level consolidation with semantic segment-level consolidation. Segment-level batching reduces construction frequency, while semantic boundary detection determines coherent segment composition before each segment is encoded into context-independent typed records. Together with

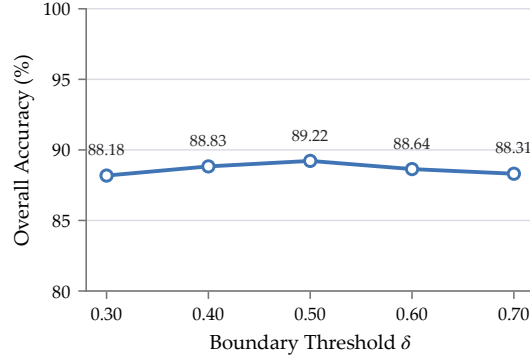


Figure 5: Boundary-threshold sensitivity on LoCoMo with GPT-4.1-Mini. We vary the cut threshold δ applied to p_t while keeping all other settings fixed. The default $\delta = 0.50$ achieves the highest accuracy, and overall accuracy varies by 1.04 percentage points across the evaluated range.

lightweight cross-segment disambiguation, structured evidence indexes, and query-planned multi-route recall, this design preserves fine-grained conversational evidence while avoiding unnecessary construction and query-time overhead. Experiments on LoCoMo and LongMemEval-S demonstrate that this design improves long-term memory QA and yields a stronger accuracy–cost trade-off than representative memory baselines. Future work will extend this segment-level evidence construction paradigm beyond text-only conversational QA toward multimodal memories, richer preference modeling, and deployment-oriented memory governance.

References

- Seyed Moein Abtahi, Rasa Rahnema, Hetkumar Patel, Neel Patel, Majid Fekri, and Tara Khani. Memanto: Typed semantic memory with information-theoretic retrieval for long-horizon agents, 2026. URL <https://arxiv.org/abs/2604.22085>.
- Jiale Chang and Yuxiang Ren. Scrapmem: A bio-inspired framework for on-device personalized agent memory via optical forgetting, 2026. URL <https://arxiv.org/abs/2605.03804>.
- Jiayi Chen, Yingcong Li, and Guiling Wang. Memflow: Intent-driven memory orchestration for small language model agents, 2026a. URL <https://arxiv.org/abs/2605.03312>.
- Yaoqi Chen, Haibin Lai, Yuru Feng, Chuyu Han, Qianxi Zhang, Baotong Lu, Menghao Li, Xinjiang Wang, Zhirui Wang, Shusen Xu, Zengzhong Li, Zewen Jin, Hao Wu, Cheng Li, and Qi Chen. Beyond semantic organization: Memory as execution state management for long-horizon agents, 2026b. URL <https://arxiv.org/abs/2606.06090>.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. *CoRR*, abs/2504.19413, 2025.
- Minjun Choi, Yoonjin Jang, Sangwon Youn, and Youngjoong Ko. G-long: Graph-enhanced memory management for efficient long-term dialogue agents, 2026. URL <https://arxiv.org/abs/2606.13115>.
- Jizhan Fang, Xinle Deng, Haoming Xu, Ziyang Jiang, Yuqi Tang, Ziwen Xu, Shumin Deng, Yunzhi Yao, Mengru Wang, Shuofei Qiao, Huajun Chen, and Ningyu Zhang. Lightmem: Lightweight and efficient memory-augmented generation. *CoRR*, abs/2510.18866, 2025.
- Hao-Lun Hsu, Nikki Lijing Kuang, Boyi Liu, Zhewei Yao, and Yuxiong He. Organize then retrieve: Hierarchical memory navigation for efficient agents, 2026. URL <https://arxiv.org/abs/2606.11680>.
- Tianyu Hu, Weikai Lin, Weizhi Zhang, Jing Ma, and Song Wang. Memrouter: Memory-as-embedding routing for long-term conversational agents, 2026. URL <https://arxiv.org/abs/2605.00356>.

- Suozhao Ji, Baodong Wu, Zehao Wang, Lei Xia, Qingping Li, Ruisong Wang, Wenbo Ding, Zhenhua Zhu, Boxun Li, Guohao Dai, and Yu Wang. Infini memory: Maintainable topic documents for long-term llm agent memory, 2026. URL <https://arxiv.org/abs/2606.10677>.
- Yunhan Jiang, Wenbin Duan, Shasha Guo, Liang Pang, Xiaoqian Sun, and Huawei Shen. Activemem: Distributed active memory for long-horizon llm reasoning, 2026. URL <https://arxiv.org/abs/2606.10532>.
- Jiazheng Kang, Mingming Ji, Zhe Zhao, and Ting Bai. Memory OS of AI agent. *CoRR*, abs/2506.06326, 2025.
- Minjae Kim, Jinheon Baek, Soyeong Jeong, and Sung Ju Hwang. Memrefine: Llm-guided compression for long-term agent memory, 2026. URL <https://arxiv.org/abs/2606.13177>.
- Chunyu Li, Mengyuan Zhang, Jingyi Kang, Ding Chen, Jiajun Shen, Bo Tang, Xuanhe Zhou, Feiyu Xiong, and Zhiyu Li. Memreranker: Reasoning-aware reranking for agent memory retrieval, 2026a. URL <https://arxiv.org/abs/2605.06132>.
- Kai Li, Xuanqing Yu, Ziyi Ni, Yi Zeng, Yao Xu, Zheqing Zhang, Xin Li, Jitao Sang, Xiaogang Duan, Xuelei Wang, Chengbao Liu, and Jie Tan. Timem: Temporal-hierarchical memory consolidation for long-horizon conversational agents, 2026b. URL <https://arxiv.org/abs/2601.02845>.
- Yilong Li, Suman Banerjee, and Tong Che. Ember: Efficient memory via budgeted evidence retention for long-horizon agents, 2026c. URL <https://arxiv.org/abs/2606.05894>.
- Yuyang Li, Yime He, Zeyu Zhang, and Dong Gong. Evimem: Evidence-gap-driven iterative retrieval for long-term conversational memory, 2026d. URL <https://arxiv.org/abs/2604.27695>.
- Zhiyu Li, Shichao Song, Hanyu Wang, Simin Niu, Ding Chen, Jiawei Yang, Chenyang Xi, Huayi Lai, Jihao Zhao, Yezhaohui Wang, Junpeng Ren, Zehao Lin, Jiahao Huo, Tianyi Chen, Kai Chen, Kehang Li, Zhiqiang Yin, Qingchen Yu, Bo Tang, Hongkang Yang, Zhi-Qin John Xu, and Feiyu Xiong. Memos: An operating system for memory-augmented generation (MAG) in large language models. *CoRR*, abs/2505.22101, 2025. URL <https://doi.org/10.48550/arXiv.2505.22101>.
- Keer Lu, Liwei Chen, Guoqing Jiang, Zhiheng Qin, Yunhuai Liu, and Wentao Zhang. Real: A reasoning-enhanced graph framework for long-term memory management of llms, 2026.
- Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of llm agents, 2024. URL <https://arxiv.org/abs/2402.17753>.
- Memobase, Inc. Memobase: User profile-based long-term memory for ai chatbot applications. <https://github.com/memodb-io/memobase>, 2025. Accessed: 2025-01-04.
- Jiayan Nan, Wenquan Ma, Wenlong Wu, and Yize Chen. Nemori: Self-organizing agent memory inspired by cognitive science. *CoRR*, abs/2508.03341, 2025. doi: 10.48550/ARXIV.2508.03341. URL <https://doi.org/10.48550/arXiv.2508.03341>.
- NevaMind-AI. memU: Memory Infrastructure for LLMs and AI Agents, 2025. URL <https://github.com/NevaMind-AI/memU>. GitHub repository, accessed 2026-07-04.
- Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems. *CoRR*, abs/2310.08560, 2023. URL <https://doi.org/10.48550/arXiv.2310.08560>.
- Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Xufang Luo, Hao Cheng, Dongsheng Li, Yuqing Yang, Chin-Yew Lin, H. Vicky Zhao, Lili Qiu, and Jianfeng Gao. Secom: On memory construction and retrieval for personalized conversational agents. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025. URL <https://openreview.net/forum?id=xKDZAW0He3>.
- Matteo Stabile and Enrico Zimuel. Dmf: A deterministic memory framework for conversational ai agents, 2026. URL <https://arxiv.org/abs/2606.03463>.

- Ziming Wang. Toki: A bitemporal operator algebra for contradiction resolution in llm-agent persistent memory, 2026. URL <https://arxiv.org/abs/2606.06240>.
- Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking chat assistants on long-term interactive memory. In *ICLR*. OpenReview.net, 2025. URL <https://openreview.net/forum?id=pZiyCaVuti>.
- Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-MEM: agentic memory for LLM agents. *CoRR*, abs/2502.12110, 2025. doi: 10.48550/ARXIV.2502.12110. URL <https://doi.org/10.48550/arXiv.2502.12110>.
- Ningning Zhang, Xingxing Yang, Zhizhong Tan, Weiping Deng, and Wenyong Wang. Himem: Hierarchical long-term memory for LLM long-horizon agents. *CoRR*, abs/2601.06377, 2026. doi: 10.48550/ARXIV.2601.06377. URL <https://doi.org/10.48550/arXiv.2601.06377>.
- Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. Memorybank: Enhancing large language models with long-term memory. In *AAAI*, pp. 19724–19731, 2024. URL <https://doi.org/10.1609/aaai.v38i17.29946>.

Table 6: LoCoMo question categories used in the main evaluation. Counts sum to the 1,540 valid QA pairs used for all LoCoMo accuracy and cost analyses.

Question type	# Questions	Evidence pattern	Capability stressed
Single-hop	841	A localized fact, event, preference, or detail is sufficient to answer the question.	Fine-grained fact preservation and speaker/entity disambiguation.
Multi-hop	282	Evidence must be combined across multiple turns, events, or sessions.	Cross-session evidence coverage and entity continuity.
Temporal reasoning	321	The answer depends on dates, ordering, duration, recurrence, or relative time expressions.	Temporal normalization, date-aware retrieval, and reasoning over event order.
Open-domain	96	The question is phrased more openly and may have weak lexical overlap with the relevant dialogue evidence.	Robust evidence selection under non-template queries.
Total	1,540	All retained LoCoMo QA pairs.	Overall micro-average.

A Experimental Protocol Details

This appendix specifies the evaluation protocol used in Section 4. It details the benchmark composition, answer generation protocol, judge configuration, and token accounting rules, so that the reported accuracy-cost trade-off can be interpreted without relying on hidden implementation assumptions. Unless otherwise stated, all generation and judge calls use deterministic decoding with temperature set to 0.

A.1 Benchmark Data and Evaluation Tasks

We report the benchmark statistics separately because LoCoMo and LongMemEval-S use different evaluation units and question taxonomies. LoCoMo evaluates multiple question-answer pairs over each multi-session conversation, whereas LongMemEval-S evaluates one question for each long conversation-question instance. Aggregating these statistics into a single table would obscure the different sources of difficulty in the two benchmarks.

A.1.1 LoCoMo

LoCoMo (Maharana et al., 2024) is a long-context conversational memory benchmark built around companion-style multi-session dialogues. Each dialogue contains two named speakers and requires a memory system to recover fine-grained personal facts, resolve speaker-specific references, and combine evidence distributed across turns or sessions.

Data statistics. LoCoMo is small in the number of conversations but dense in annotated memory questions. It contains 10 multi-session conversations, with each conversation averaging approximately 600 turns and 16K tokens. The benchmark provides 1,986 question-answer pairs in total; the main evaluation retains 1,540 QA pairs from the standard long-term memory categories after excluding the adversarial/counterfactual subset.

Task categories. The LoCoMo main evaluation uses 1,540 valid question-answer pairs from the first four categories. We exclude category 5 adversarial/counterfactual questions because these examples are not part of the standard semantic-answering setting used for the main long-term memory QA comparison. The same 1,540 questions are used for the main results, cost analysis, and LoCoMo ablations, so method variants are compared on an identical evaluation set. Table 6 gives the category-level composition.

A.1.2 LongMemEval-S

LongMemEval-S (Wu et al., 2025) focuses on agent-style task-oriented interactions with substantially longer dialogue histories. In contrast to LoCoMo, each evaluation item corresponds to one long

Table 7: LongMemEval-S question categories used in the main evaluation. Counts sum to the 500 questions in the official cleaned split; abbreviations are defined in the text.

Question type	# Questions	Memory ability	Evidence requirement
SSU	70	User-fact recall	Recover a fact provided by the user within a single session.
SSA	56	Assistant-side recall	Recover information, commitments, or responses previously given by the assistant.
SSP	30	Preference tracking	Use user-specific preference information to satisfy the reference rubric.
MS	133	Cross-session reasoning	Integrate evidence distributed across multiple sessions.
KU	78	Changed-state recall	Answer with the updated state from information that changes across the dialogue history.
TR	133	Temporal reasoning	Interpret order, duration, relative time, or time-bounded evidence.
Total	500	Official cleaned split	Overall micro-average.

conversation-question instance. This setting stresses whether a system can recover user facts, assistant-side facts, preferences, cross-session evidence, changed information, and temporal relations under much higher context load.

Data statistics. LongMemEval-S is organized at the instance level rather than as multiple QA pairs over the same conversation. The official cleaned split contains 500 conversation-question instances, with one question attached to each long dialogue history. Its average context length is approximately 115K tokens per instance, making it substantially longer than LoCoMo and better suited for testing memory construction under high context load.

Task categories. The LongMemEval-S evaluation uses all 500 questions in the official cleaned split. We report category accuracy using the official question type field and compute the overall score as a micro-average over all 500 questions. For compact table layout, Table 7 abbreviates the official category names as SSU (single-session-user), SSA (single-session-assistant), SSP (single-session-preference), MS (multi-session), KU (knowledge-update), and TR (temporal-reasoning).

A.2 Answer Generation and Judge Protocol

We evaluate all methods with GPT-4.1-Mini and GPT-4o-Mini as answer models. For each model, all methods use the same benchmark split and judge protocol. Each memory system constructs or retrieves an evidence context according to its own design before the model generates the final answer. For LYCHEEMEMORY, the same model is also used for memory encoding and query planning. Predictions from both models are evaluated with the same benchmark-specific GPT-4o-Mini judge. The token-cost analysis uses GPT-4.1-Mini. Table 8 summarizes the components used by LYCHEEMEMORY.

For LoCoMo, the judge receives the question, the gold answer, and the generated answer, and returns whether the generated answer is semantically correct. The judge is instructed to accept equivalent phrasings and concise answers that refer to the same fact or time period as the gold answer. Category accuracy is computed within each retained question type, and the overall score is computed over all 1,540 retained questions.

For LongMemEval-S, we use the official task-specific yes/no judge protocol with a 10-token output cap for the judge response. Standard single-session and multi-session questions are judged by whether the generated response contains the correct answer, while preference questions are judged against a rubric-style desired response. For knowledge-update questions, the judge accepts a response that contains the updated answer even if it also mentions previous information. Temporal-reasoning questions use the temporal-specific judge rule, including the official tolerance for minor off-by-one differences in duration-style answers.

Table 8: Evaluation components used by LYCHEEMEMORY. The same model is used for memory encoding, query planning, and answer generation; the cost and ablation experiments use GPT-4.1-Mini. Temperature is set to 0 for all generative components.

Component	Model	Role
Memory encoder	GPT-4.1-Mini GPT-4o-Mini	/ Convert finalized segments into typed memory records.
Query planner	GPT-4.1-Mini GPT-4o-Mini	/ Produce typed recall routes for evidence retrieval.
Answer generator	GPT-4.1-Mini GPT-4o-Mini	/ Generate the final answer from the retrieved evidence.
Embedding model	text-embedding-3-small	Embed exchanges, records, and queries.
Reranker	bge-reranker-v2-m3	Rerank retrieved evidence candidates without generative decoding.
LoCoMo judge	GPT-4o-Mini	Judge semantic correctness for LoCoMo answers.
LongMemEval-S judge	GPT-4o-Mini	Apply the official task-specific yes/no judge.

Table 9: Metric definitions and accounting rules. Token counts include generative LLM input and output tokens and are reported in thousands (K).

Metric	Definition	Notes
Category accuracy	Number of correct answers in a category divided by the number of valid questions in that category.	Reported using each benchmark’s question taxonomy.
Overall accuracy	Number of correct answers divided by the number of valid questions in the benchmark.	Micro-average over questions, not a macro-average over categories.
Construction tokens	Generative LLM input and output tokens consumed during memory construction.	Averaged per conversation or conversation-question instance; reported in K.
Query tokens	Generative LLM input and output tokens consumed during answer generation and any query-time generative planning.	Averaged per question; reported in K.

A.3 Metrics and Token Accounting

We report category accuracy, overall accuracy, construction tokens, and query tokens. Accuracy is always computed over the valid evaluation set defined for each benchmark: 1,540 retained questions for LoCoMo and 500 questions for LongMemEval-S. No additional filtering is applied beyond these benchmark-specific valid sets. Table 9 summarizes the metric definitions and accounting rules.

Construction-token accounting covers memory-building calls such as segment-level encoding. Query-token accounting covers answer generation and the single query-planning call used by LYCHEEMEMORY. Non-generative operations, including embedding lookup, structured filtering, reciprocal-rank fusion, reranking, and diversity-aware selection, do not consume generative LLM tokens and are therefore not counted as construction or query tokens. All token costs reported in Section 4.3 use GPT-4.1-Mini. This accounting separates write-side cost from query-side cost.

B Algorithmic Description

This appendix gives algorithm-level descriptions of the two core procedures in LYCHEEMEMORY. The main text explains the design motivation, mathematical scoring components, and system modules; the pseudocode below records the computation path needed to understand when generative LLM calls occur, which intermediate states are maintained, and how the final evidence context is assembled.

Algorithm B.1: Semantic Segment-Level Memory Construction**Input:** conversation stream $C = \{x_1, \dots, x_T\}$; session identifier s ; memory store $\mathcal{M}$; boundary threshold δ .**Output:** updated memory store $\mathcal{M}$.

```

1   Initialize active segment  $S \leftarrow \emptyset$ , same-session reference context  $\rho_s \leftarrow \emptyset$ , and local surprise history  $H_s \leftarrow \emptyset$ .
2   Procedure FINALIZESEGMENT( $S$ , reason):
3       Encode  $S$  with reference context  $\rho_s$ :  $(\mathcal{R}, d) \leftarrow \text{ENCODE}(S, \rho_s)$ .
4       Normalize each  $r \in \mathcal{R}$  into (id, type, text, entities, tags, temporal, provenance).
5       Insert each normalized record into  $\mathcal{M}$ .
6       Build entity, topic, entity-topic, temporal, and event-frame evidence nodes from record metadata.
7       Update  $\rho_s$  with  $d$  and recent same-session record summaries; reset  $S \leftarrow \emptyset$ .
8   For each incoming exchange  $x_t \in C$  do:
9       Index  $x_t$  as raw dialogue evidence and embed  $x_t$  with the exchange encoder.
10      If  $S = \emptyset$ , start  $S$  with  $x_t$  and finalize it if the target length or exchange-count limit is reached.
11      Compute semantic surprise, cohesion drop, length pressure, and exchange-count pressure for  $x_t$  relative to  $S$ .
12      Combine the boundary signals into  $p_{\text{cut}}$  and update  $H_s$  with the current semantic surprise.
13      If appending  $x_t$  would exceed the hard segment capacity, call FINALIZESEGMENT( $S$ , capacity_limit) and start a new  $S$  with  $x_t$ .
14      Else if  $p_{\text{cut}} \geq \delta$ , call FINALIZESEGMENT( $S$ , semantic_boundary) and start a new  $S$  with  $x_t$ .
15      Else append  $x_t$  to  $S$  and finalize  $S$  if the target length or exchange-count limit is reached.
16  End for. If the session is flushed and  $S \neq \emptyset$ , call FINALIZESEGMENT( $S$ , session_flush).
17  Return  $\mathcal{M}$ .

```

B.1 Semantic Segment-Level Memory Construction

Algorithm B.1 corresponds to Sections 3.2–3.4. It emphasizes three implementation properties: segment-level batching reduces construction frequency by assigning one encoding call to each finalized segment rather than each exchange; semantic boundary detection uses embeddings and deterministic scoring to determine segment composition; and cross-segment disambiguation is carried forward only as reference context, not as a new source of facts.

B.2 Plan-Guided Multi-Route Retrieval

Algorithm B.2 corresponds to Section 3.5. It uses one LLM planning call to convert the question into typed evidence routes, while route recall, temporal filtering, record search, raw-turn search, reranking, reciprocal-rank fusion, and diversity-aware selection are non-generative operations.

C Implementation Details

This appendix provides the default implementation parameters following the pipeline order in Section 3. The four stages are online semantic segmentation, segment-level memory encoding, structured evidence organization, and plan-guided multi-route retrieval.

C.1 Online Semantic Segmentation

Online semantic segmentation decides when a buffered dialogue segment should be encoded by the memory encoder. Unlike turn-level eager consolidation, segment-level batching allows multiple exchanges to share one encoding call and therefore controls construction frequency. Within this batching scheme, the semantic boundary policy determines which exchanges form each segment: LYCHEEMEMORY finalizes the active segment when it becomes saturated, a topic transition is detected, or a hard capacity limit is reached. Boundary scoring uses embeddings and deterministic computation only.

Each exchange is embedded with `text-embedding-3-small`. The system maintains the active segment centroid, the most recent exchange embedding, the local semantic-surprise history, the current token length, and the exchange count. Table 10 lists the default segmentation parameters. The implementation converts the combined boundary score into a cut probability:

Algorithm B.2: Plan-Guided Multi-Route Retrieval

Input: user query q ; recent dialogue context H_q ; memory store $\mathcal{M}$; requested evidence budget k ; configured planning depth k_{plan} .

Output: evidence context E_q for answer generation.

```

1  Generate a structured retrieval plan with one LLM call:  $\Pi \leftarrow \text{PLAN}(q, H_q)$ .
2  Normalize  $\Pi$  and select a retrieval strategy from the planned question type.
3  Set the effective evidence budget  $k \leftarrow \max(k, k_{\text{plan}}, 1)$ .
4  For each evidence route  $R_i \in \Pi.\text{routes}$  do:
5      Build route-specific query variants from  $q$  and  $R_i$ ; initialize candidate set  $D_i \leftarrow \emptyset$ .
6      If  $R_i$  or  $\Pi$  specifies a temporal filter, retrieve matching records through temporal evidence nodes
       and add them to  $D_i$ .
7      For each query variant  $v$  in  $R_i$  do:
8          Retrieve entity, topic, entity-topic, and event-frame evidence nodes by semantic search.
9          Expand matched evidence nodes to linked memory records and add them to  $D_i$ .
10         Retrieve memory records by semantic search over record embeddings and add direct hits to
             $D_i$ .
11         Retrieve raw dialogue turns by semantic search and add raw-turn hits to  $D_i$ .
12         Merge duplicate candidates inside  $D_i$ .
13         Apply question-type-specific score adjustments and local evidence expansion.
14         Optionally rerank  $D_i$  with a non-generative cross-encoder reranker.
15         Sort  $D_i$  to obtain a route-level ranked list  $L_i$ .
16     End for. Fuse route-level lists  $\{L_1, \dots, L_m\}$  with reciprocal-rank fusion.
17     Preserve route coverage, apply diversity-aware selection, and enforce temporal/source-type budget
        controls.
18     Serialize selected memory records and raw dialogue snippets into  $E_q$ .
19     Return  $E_q$ .
```

Table 10: Default segmentation parameters. The segmentation step uses no generative LLM calls.

Parameter / signal	Value	Function
Exchange embedding model	text-embedding-3-small	Embed each incoming exchange.
Cut probability threshold	0.50	Finalize the active segment when the cut probability reaches this threshold.
Surprise history window	64 values	Support local normalization of semantic surprise.
Minimum history for robust signal	5 values	Disable robust surprise normalization when the history is too short.
Robust surprise normalization	median/MAD z-score clipped to [-2.0, 4.0]	Reduce outlier and early-stage noise.
Absolute surprise signal	$\text{clip}((s - 0.20) / 0.14, -1.0, 2.5)$	Provide an absolute novelty component.
Minimum chunk tokens	300	Avoid excessive fragmentation.
Target chunk tokens	600	Preferred consolidation scale.
Maximum chunk tokens	900	Hard token cap.
Maximum exchanges	10	Hard exchange-count cap.

$$P_{\text{cut}} = \sigma(-1.10 + \text{score}), \quad \text{cut if } P_{\text{cut}} \geq 0.50.$$

The length signal is a piecewise pressure function over the current segment length. It is set to -1.30 below $0.7 * \text{min_tokens}$, grows from -0.80 to 0 before the minimum length, grows from 0.45 to 1.90 between the minimum and target lengths, grows from 1.90 to 2.80 between the target and maximum lengths, and becomes 3.00 at or beyond the maximum length. The turn-count signal is -0.85 for one exchange, -0.15 for two exchanges, 0.15 for three exchanges, and $\min(1.0, 0.30 + 0.15 * (n - 4))$ for four or more exchanges. If appending a new exchange would exceed the maximum token cap, the current segment is finalized before the new exchange is added.

Table 11: Segment-level memory encoding settings.

Item	Setting
Encoder model	GPT-4.1-Mini or GPT-4o-Mini, matching the answer model
Temperature	0
Max output tokens	Provider default
Output format	Raw JSON object without markdown fences
disambiguation_context budget	1,200 characters
Same-session reference context budget	2,400 characters
Recent semantic records retained	12

Table 12: Memory record fields produced by the segment encoder.

Field	Description
record_id	Internal identifier used to link storage and retrieval entries.
memory_type	One of <code>fact</code> , <code>preference</code> , <code>event</code> , <code>constraint</code> , <code>procedure</code> , <code>failure_pattern</code> , or <code>tool_affordance</code> .
semantic_text	Self-contained natural-language memory statement.
normalized_text	Concatenation of memory type and semantic text.
entities	Canonical entity names.
tags	Topic, action, or type tags.
temporal	Normalized event or validity times, represented by fields such as <code>t_ref</code> , <code>t_valid_from</code> , and <code>t_valid_to</code> .
evidence_turn_range	Source turn indexes.
source_session	Source session identifier.
source_role	<code>user</code> , <code>assistant</code> , <code>both</code> , or empty.
confidence	Current implementation uses 1.0.

C.2 Segment-Level Memory Encoding

Segment-level memory encoding converts each finalized segment into typed records that can be retrieved and interpreted without the original dialogue context. The encoder receives the current segment text and a compact same-session reference context, then returns memory records and an updated disambiguation state. Tables 11 and 12 summarize the encoding settings and record schema.

C.3 Structured Evidence Organization

Structured evidence organization converts metadata from the segment encoder into searchable evidence nodes. In addition to record embeddings, LYCHEEMEMORY organizes records into entity, topic, entity-topic, temporal, and event-frame indexes. This phase does not call a generative LLM; it uses embedding, SQLite/FTS indexing, and deterministic bookkeeping. Table 13 summarizes the organization settings.

LYCHEEMEMORY retains textually distinct statements as separate records, including records that may describe successive or conflicting states. The organizer does not perform approximate semantic merging, infer supersession links, close the validity interval of an earlier record, or automatically expire it when a later statement is inserted. This preserves the available historical evidence while leaving conflict resolution to retrieval and answer generation.

C.4 Plan-Guided Multi-Route Retrieval

Plan-guided multi-route retrieval uses one LLM planning call for query understanding and then executes non-generative evidence collection. The planner’s internal question types are `single`, `aggregate`, `temporal`, `comparison`, `personalized_advice`, `prior_assistant_response`, and `other`. These types are not benchmark categories; they select retrieval strategies, candidate budgets, and route constraints. Table 14 lists the default retrieval parameters.

Table 13: Structured evidence organization settings.

Component	Implementation	Notes
Vector backend	LanceDB	Stores record embeddings for direct semantic retrieval.
Structured store	SQLite + FTS5	Stores evidence nodes and full-text searchable fields.
Entity/tag normalization	Case folding, separator normalization, punctuation cleanup, repeated-whitespace merge	Produces stable keys.
Temporal nodes	Day-level and month-level keys	Derived from <code>t_ref</code> , <code>t_valid_from</code> , and <code>t_valid_to</code> .
Event-frame index	Finalized semantic segment	Each node groups the records produced from one finalized semantic segment and retains its source session and turn range as provenance.
Evidence-node merge	<code>node_type:key</code>	Merges repeated entity, topic, and temporal nodes.

Table 14: Default retrieval parameters.

Retrieval parameter	Value
Planner model	GPT-4.1-Mini or GPT-4o-Mini, matching the answer model; temperature 0
Final top-k	$\max(\text{requested_top_k}, \text{plan_depth}, 1)$
<code>plan_depth</code>	15
Evidence-node recall budget	$\max(\text{top_k} * 3, 30) * \text{evidence_limit_multiplier}$
Direct record recall budget	$\max(\text{top_k} * 3, 30) * \text{record_limit_multiplier}$
Raw-turn recall budget	$\max(\text{top_k}, 20) * \text{turn_limit_multiplier}$
Temporal filter recall budget	$\max(\text{top_k} * 10, 100)$
RRF smoothing constant	60.0
Reranker	bge-reranker-v2-m3
Route-level rerank candidate limit	$\max(\text{configured_reranker_candidate_limit}, \text{top_k} * 4)$
Configured reranker candidate limit	100

Each route can activate direct record recall, evidence-node recall, temporal recall, and raw-turn recall. Route-level candidates are merged with reciprocal-rank fusion using a smoothing constant of 60.0. After fusion, the system first preserves route coverage using the route quota $\max(1, \min(4, \text{top_k} // \text{route_count}))$. The final candidate score is:

$$s_{\text{final}} = 0.82 s_{\text{best}} + 0.18 s_{\text{route}}.$$

Diversity-aware selection uses an MMR-style objective:

$$s_{\text{select}} = 0.75 s_{\text{rel}} - 0.25 s_{\text{sig}}.$$

The candidate signature contains the source session, entities, matched queries, evidence nodes, turns, and text tokens. The final evidence budget is count-based top-k rather than a fixed token budget, so query prompt length varies with the selected evidence text. This is why Section 4.3 reports empirical average query tokens.

D Token Accounting Details

This appendix expands the token-accounting definitions in Appendix A.3. The distinction between construction tokens and query tokens is central to the paper’s conclusion: LYCHEEMEMORY aims to reduce write-side construction cost without shifting the cost to query-time context expansion.

D.1 Construction Tokens

Construction tokens count all recorded generative LLM input and output tokens consumed during memory building, averaged by conversation or conversation-question instance and reported in thousands (K):

$$\text{ConstructionTokens} = \frac{\sum_{c \in C_{\text{build}}} (t_c^{\text{in}} + t_c^{\text{out}})}{N_{\text{conv}}}.$$

For LYCHEEMEMORY, construction tokens include all recorded memory-encoding input and output tokens, covering the segment memory encoding prompt, finalized segment text, reference/disambiguation context, memory-record output, and disambiguation-state output. Construction tokens exclude embedding computation, deterministic structured indexing, SQLite/FTS/vector operations, BM25 or vector search, non-generative reranking, evaluation judge calls, and final answer generation calls.

D.2 Query Tokens

Query tokens count all recorded generative LLM input and output tokens consumed while answering evaluation questions, averaged by question and reported in thousands (K):

$$\text{QueryTokens} = \frac{\sum_{q \in Q} (t_q^{\text{in}} + t_q^{\text{out}})}{|Q|}.$$

For LYCHEEMEMORY, query tokens include the query-planner input/output, final answer-generation input/output, serialized evidence context, and recent dialogue context included in the answer prompt. Query tokens exclude embedding retrieval, structured filtering, reciprocal-rank fusion, non-generative cross-encoder reranking, diversity-aware selection, and evaluation judge calls. Because the final evidence context is count-based rather than token-budget based, query tokens reflect the average prompt length after selected evidence has been serialized.

E Ablation Variant Definitions

This appendix defines the implementation of each ablation variant in Section 4.4. The main text discusses the results, while the appendix specifies which modules are removed, replaced, or preserved in each comparison.

E.1 Construction-Side Variants

per-turn construction (w/o segment-level batching). This variant removes segment-level batching and falls back to turn-level eager construction. On LoCoMo, it triggers memory construction at the speaker-turn ingestion granularity; on LongMemEval-S, it uses the original message/exchange granularity. The record schema, structured indexing, and query-time retrieval pipeline remain unchanged. This variant evaluates the effect of replacing segment-level batching with eager construction.

fixed-window consolidation. This variant replaces semantic boundary detection with fixed-window batching. It preserves batching, but segment boundaries no longer depend on semantic surprise or cohesion drop. The default configuration uses 600 target chunk tokens and at most 10 exchanges. This variant evaluates semantic boundary detection relative to mechanical windows at a comparable batching scale.

E.2 Representation-Side Variants

summary-level records. This variant replaces typed, self-contained records with a summary-level representation. It no longer explicitly retains the typed atomic structure formed by memory type, entity, topic, temporal scope, and provenance fields, resulting in summary-level retrieval without

these structured fields. The rest of the retrieval pipeline remains unchanged, enabling a comparison between typed records and summary-level memory.

w/o cross-segment reference context. This variant encodes each segment independently and does not pass same-session disambiguation context or recent resolved records to subsequent segments. The record schema and retrieval pipeline remain unchanged. This variant evaluates the cross-segment context configuration.

E.3 Retrieval-Side Variants

record-vector retrieval only. This variant disables entity, topic, temporal, event-frame, and raw-turn multi-route recall, and uses only memory-record vector search to return evidence candidates. The final evidence budget is kept consistent with the requested top- k of the full system. This variant evaluates the combined structured and raw-turn recall channels relative to memory-record vector search.

w/o query planner. This variant removes the LLM query planner and constructs a fixed single-route query directly from the original question. It performs no LLM query rewriting, question-type classification, or route decomposition. Query tokens therefore decrease, but retrieval routes cannot adapt to temporal, comparison, personalized-advice, or prior-assistant-response intents.

w/o fusion/reranking/diversity selection. This variant preserves recall candidates but jointly removes reciprocal-rank fusion, cross-encoder reranking, and diversity-aware selection. Candidates are sorted by their original retrieval or field scores and truncated to top- k . This variant evaluates the combined evidence-selection stack.

F Prompt Templates and Evaluation Prompts

This appendix reports the prompt templates used by LYCHEEMEMORY and the evaluation prompts used for judging. The LYCHEEMEMORY memory encoding, query planning, and answer generation prompts are taken from the implementation files `src/memory/semantic/prompts.py` and `src/core/semantic_pipeline.py`. The LongMemEval-S judge prompts follow the official task-specific yes/no evaluator, and the LoCoMo judge prompt follows the JSON label protocol used in the LoCoMo evaluation implementation. Prompt blocks below are line-wrapped for typesetting.

F.1 LYCHEEMEMORY Segment Encoding Prompt

Segment encoding uses a system message and a user message. The user message contains `<SESSION_DATE>`, `<REFERENCE_CONTEXT>`, and `<CURRENT_TURNS>`. The reference context is used only for resolving references and aliases; it is not treated as a source from which new facts may be extracted.

Segment Encoding Prompt — System Message

You are a memory encoder. Given a conversation chunk, extract atomic memory records that are self-contained and retrievable without the original dialogue.

Input

- `<SESSION_DATE>`: date anchor for converting relative dates. May be absent.
- `<REFERENCE_CONTEXT>`: notes from earlier chunks in the same session. Use only to resolve references; never extract facts from it.
- `<CURRENT_TURNS>`: the turns to extract from. `evidence_turns` are 0-based indexes into this block.

Rules

1. One fact per record. Write each `semantic_text` as a standalone sentence - replace every pronoun, alias, and implicit subject with the explicit referent. Preserve exact names, quantities, dates, places, reasons, and outcomes.

2. Convert relative dates ("yesterday", "next Monday") to absolute dates (YYYY-MM-DD) in both `semantic_text` and `temporal` when `<SESSION_DATE>` is provided.
3. Conversation chunks may contain explicit speaker prefixes inside the content, such as "Gina: ..." or "Jon: ...", even when the outer turn role is `user`. Treat those prefixed names as the true speakers.
4. When a named speaker says "I", "me", or "my", resolve it to that speaker's name. Do not rewrite named speakers as "the user". Use "the user" only when no speaker name is available.
5. Store durable facts, preferences, plans, constraints, and events stated by any real conversation speaker. Skip greetings, filler, acknowledgements, generic encouragement, and generic world knowledge.
6. Store assistant content only when it is a substantive remembered part of the conversation; skip ingestion acknowledgements such as "OK" or "Acknowledged".
7. Return `disambiguation_context` as a short note listing resolved speaker names, aliases, relationships, and dates needed for later reference resolution. Empty string if none.
8. If no durable information exists, return `{"records": [], "disambiguation_context": ""}`.

Output - raw JSON, no markdown

```
{
  "records": [
    {
      "memory_type":
        "fact | preference | event | constraint | procedure |
        failure_pattern | tool_affordance",
      "semantic_text": "complete standalone sentence",
      "entities": ["searchable named entity"],
      "temporal": {"t_ref": "", "t_valid_from": "", "t_valid_to": ""},
      "tags": ["short keyword"],
      "evidence_turns": [0],
      "source_role": "user | assistant | both"
    }
  ],
  "disambiguation_context": ""
}
```

Memory types: fact (stable attribute/relationship/state); preference (like/dislike/habit/recurring choice); event (dated one-off occurrence); constraint (limit/policy/restriction); procedure (reusable steps/workflow); failure_pattern (mistake/pitfall/lesson); tool_affordance (tool capability or limitation).

Segment Encoding Prompt — User Message

```
<SESSION_DATE>{session_date}</SESSION_DATE>
<REFERENCE_CONTEXT>
{reference_context}
</REFERENCE_CONTEXT>

<CURRENT_TURNS>
{current_turns}
</CURRENT_TURNS>
```

F.2 LYCHEEMEMORY Query Planning Prompt

The query planning prompt converts the current question into an executable retrieval plan. The planner describes only the evidence need visible in the user question and recent context; it does not decide whether the answer exists and does not invent answer candidates. For ordinary named-speaker factual questions, the default behavior is a single route; multiple routes are used only when the question explicitly requires separate evidence.

Query Planning Prompt — System Message

Write a JSON plan for answering a user question from past conversation memory.

Inputs:

- `<USER_QUERY>`: current request.
- `<RECENT_CONTEXT>`: recent turns, if any.

Your job:

- Look only at the user-visible wording in `<USER_QUERY>` and `<RECENT_CONTEXT>`.
- Describe only what the user is asking for.
- Do not decide whether the answer is available or unavailable.
- Do not speculate beyond the user-visible question.

Question classification:

- `question_type` describes the question form visible from the request: `single`, `aggregate`, `temporal`, `comparison`, `personalized_advice`, `prior_assistant_response`, or `other`.
- Some questions ask about one named speaker in a two-speaker conversation. Use `single` for ordinary who / what / where / why / how factual questions about a speaker, event, preference, plan, object, or image detail.
- Use `temporal` only when the final answer needs a date, approximate date, order, duration, recurrence, or a relative-time conversion.
- Use `comparison` only when the question directly compares two or more non-time attributes.
- Use `aggregate` only when the question asks for a count, list, set, total, or all matching items.
- Use `personalized_advice` only for advice or recommendation questions.
- Use `prior_assistant_response` only when the question asks what the assistant previously said.

Query generation:

- Produce 3-8 short semantic queries. Prefer high-precision phrases over broad paraphrases.
- Preserve exact speaker names, people, places, and object names from the question. Do not rewrite named speakers as "the user".
- Include the named speaker with the key predicate in most queries.
- Add 1-2 natural paraphrases for likely conversation wording, but do not invent answer entities.
- Use the same language as the user query when natural.

Evidence target and constraints:

- `evidence_target` is the single fact/event/detail/set needed to answer the question.
- `evidence_constraints` should stay short: named speaker, key predicate, requested attribute, and any explicit time/status condition.
- Do not add generic hidden requirements that are not visible in the user question.

Evidence routes:

- Use one `evidence_routes` item by default. The route should retrieve the most relevant dialogue span or memory for the named speaker and predicate.
- Use multiple routes only when the question explicitly needs separate evidence: two named speakers, two events being compared, two endpoints of a duration, or a count/list over several members.
- Do not create separate routes for generic helper facts such as "age basis", "background", "current state", or "candidate events" unless the question explicitly requires them.
- Each route should have a concise `evidence_goal`, 3-5 route-specific `queries`, optional `constraints`, and optional `temporal_filter`.

Time:

- Set `temporal_filter` only when the request or recent context contains a concrete date or an unambiguous range that can be converted to YYYY-MM-DD.
- For vague time wording such as "recently", "earlier", "later", "next month", or "last week", usually leave `temporal_filter` empty and rely on retrieved conversation timestamps.
- For "on DATE", set both `since` and `until` to DATE. For "before DATE", set only `until`. For "after DATE" or "since DATE", set only `since`.

Self-check before returning:

- The plan should be compact. If a simple named-speaker question has more than one route, merge it.
- If the queries could match the wrong speaker, add the correct speaker name to them.
- If the plan contains invented answer candidates, remove them.

Return raw JSON only:

```
{
  "reason": "brief planning reason: what the user asks, target/constraints,
            query strategy",
  "question_type":
    "single|aggregate|temporal|comparison|personalized_advice|
    prior_assistant_response|other",
  "semantic_queries": ["query"],
  "temporal_filter": {},
  "evidence_target": "",
  "evidence_constraints": [],
  "constraints": [
    {"kind": "time|entity|status|relation|attribute|other", "value": ""}
  ],
  "evidence_routes": [
    {
      "route_id": "r1",
      "evidence_goal": "specific user-visible evidence this route should find",
      "queries": ["route-specific query"],
      "constraints": [
        {"kind": "time|entity|status|relation|attribute|other", "value": ""}
      ],
      "temporal_filter": {}
    }
  ]
}
```

F.3 LYCHEEMEMORY Answer Generation Prompt

The answer generation prompt receives two evidence blocks, episodic/semantic memories and raw memories. The prompt instructs the model to avoid double-counting overlapping evidence, answer only about the named person when a question names a person, and prioritize the most recent supported information when memories conflict.

Answer Generation Prompt — System Message

You are an intelligent memory assistant tasked with answering questions using conversation memories.

CONTEXT

You have access to memories from two speakers in a conversation. These memories are timestamped and may be relevant to the question.

There are two memory blocks:

1. Episodic/Semantic Memories: refined summaries and concise, fact-like pieces extracted from conversations.
2. Raw Memories: unprocessed conversation snippets between the two speakers.

Context Rules:

1. Episodic/Semantic memories and Raw memories may overlap in content. Avoid double-counting redundant evidence.
2. Carefully analyze both memory blocks and identify information that is actually useful for answering the question.
3. Memories are sorted by relevance.
4. The conversation may involve two named speakers. When the question names a person, answer about that person only and do not substitute facts about the other speaker.
5. In Raw Memories, speaker prefixes inside the text, such as "Gina:" or "Jon:", identify the true speaker. Use those names to resolve "I", "me", and "my".
6. A **Date**: line on an Episodic/Semantic memory is the resolved event or valid date for that memory.
7. A **Mentioned**: line on a Raw Memory is the conversation date when that raw snippet was said.

INSTRUCTIONS

1. Carefully read all provided memories.
2. Pay close attention to timestamps when time is relevant.
3. If the question asks about a specific event or fact, look for direct, explicit evidence in the memories.
4. If the question asks for advice, recommendations, or what kind of response the user would prefer, first identify the named person's preferences, habits, constraints, or past actions, then base the suggestion primarily on these person-specific signals.
5. If memories contain contradictory information, prioritize the most recent memory.
6. For time references, convert them into concrete dates based on the memory timestamp.
7. In raw memories, **Mentioned**: marks the conversation time. Do not confuse the time of conversation with the time when an event actually happened.
8. Do not say "no information found" if there are related memories that can reasonably guide a personalized answer. Only abstain when there is truly no relevant evidence.
9. Use exact words or a short phrase from the memories when possible.
10. The final answer should usually be no more than 5-6 words. Do not include explanation unless needed to avoid ambiguity.

APPROACH (Think step by step internally)

1. Identify whether the question is factual or asks for advice/preference.
2. Retrieve all memories that are clearly related to the question.
3. Check timestamps and content to locate the most reliable information.
4. For factual queries, pinpoint explicit mentions that answer the question.
5. For advice/preference queries, anchor the answer in person-specific facts.
6. If temporal reasoning or calculation is needed, do it internally and convert the result into a concrete date or time span.
7. Formulate a precise, concise answer supported by the memories.
8. Output only the answer phrase.

{time_basis}

Episodic/Semantic Memories:

{episodic_semantic_memories}

Raw Memories:

{raw_memories}

Question: {question}

Answer:

F.4 LongMemEval-S Official Judge Prompts

The LongMemEval-S evaluator constructs task-specific judge prompts with `get_anscheck_prompt(task, question, answer, response, abstention=False)`. We use GPT-4o-Mini as the metric model, temperature 0, and a 10-token output cap; a response containing “yes” is parsed as correct.

LongMemEval-S Judge Prompt — Standard

I will give you a question, a correct answer, and a response from a model. Please answer yes if the response contains the correct answer. Otherwise, answer no. If the response is equivalent to the correct answer or contains all the intermediate steps to get the correct answer, you should also answer yes. If the response only contains a subset of the information required by the answer, answer no.

Question: {question}

Correct Answer: {answer}

Model Response: {response}

Is the model response correct? Answer yes or no only.

LongMemEval-S Judge Prompt — Temporal Reasoning

I will give you a question, a correct answer, and a response from a model. Please answer yes if the response contains the correct answer. Otherwise, answer no. If the response is equivalent to the correct answer or contains all the intermediate steps to get the correct answer, you should also answer yes. If the response only contains a subset of the information required by the answer, answer no. In addition, do not penalize off-by-one errors for the number of days. If the question asks for the number of days/weeks/months, etc., and the model makes off-by-one errors, the model's response is still correct.

Question: {question}

Correct Answer: {answer}

Model Response: {response}

Is the model response correct? Answer yes or no only.

LongMemEval-S Judge Prompt — Knowledge Update

I will give you a question, a correct answer, and a response from a model. Please answer yes if the response contains the correct answer. Otherwise, answer no. If the response contains some previous information along with an updated answer, the response should be considered as correct as long as the updated answer is the required answer.

Question: {question}

Correct Answer: {answer}

Model Response: {response}

Is the model response correct? Answer yes or no only.

LongMemEval-S Judge Prompt — Preference

I will give you a question, a rubric for desired personalized response, and a response from a model. Please answer yes if the response satisfies the desired response. Otherwise, answer no. The model does not need to reflect all the points in the rubric. The response is correct as long as it recalls and utilizes the user's personal information correctly.

Question: {question}

Rubric: {answer}

Model Response: {response}

Is the model response correct? Answer yes or no only.

LongMemEval-S Judge Prompt — Abstention

I will give you an unanswerable question, an explanation, and a response from a model. Please answer yes if the model correctly identifies the question as unanswerable. The model could say that the information is incomplete, or some other information is given but the asked information is not.

Question: {question}

Explanation: {answer}

Model Response: {response}

Does the model correctly identify the question as unanswerable? Answer yes or no only.

F.5 LoCoMo Accuracy Metric and Judge Prompt

LoCoMo uses accuracy as the primary metric. For each question, GPT-4o-Mini judges whether the generated answer is semantically consistent with the gold answer. Samples labeled CORRECT count as correct; category and overall accuracy are then computed over the retained valid questions.

LoCoMo Judge Prompt — System Message

You are an expert grader that determines if answers to questions match a gold standard answer.

LoCoMo Judge Prompt — User Message

Your task is to label an answer to a question as 'CORRECT' or 'WRONG'. You will be given the following data:

1. a question (posed by one user to another user),
2. a 'gold' (ground truth) answer,
3. a generated answer which you will score as CORRECT/WRONG.

The point of the question is to ask about something one user should know about the other user based on their prior conversations. The gold answer will usually be a concise and short answer that includes the referenced topic, for example:

Question: Do you remember what I got the last time I went to Hawaii?

Gold answer: A shell necklace

The generated answer might be much longer, but you should be generous with your grading - as long as it touches on the same topic as the gold answer, it should be counted as CORRECT.

For time related questions, the gold answer will be a specific date, month, year, etc. The generated answer might be much longer or use relative time references, but you should be generous with your grading - as long as it refers to the same date or time period as the gold answer, it should be counted as CORRECT. Even if the format differs, consider it CORRECT if it is the same date.

Now it's time for the real question:

Question: {question}

Gold answer: {ground_truth}

Generated answer: {answer}

First, provide a short (one sentence) explanation of your reasoning, then finish with CORRECT or WRONG.

Output strictly in JSON format:

```
{
  "label": "CORRECT" | "WRONG",
  "reasoning": "<short explanation>"
}
```

G Limitations and Artifact Use

Our evaluation focuses on text-only long-term conversational memory. It does not cover multi-modal memories, production latency, long-term online user feedback, privacy governance, cache behavior, or storage growth under continuous deployment. LYCHEEMEMORY primarily optimizes

construction-token cost and query-time token overhead; database latency, embedding-index storage, and production monitoring require separate deployment-oriented evaluation.

The main results show a remaining weakness on preference-intensive questions. On LongMemEval-S, LYCHEEMEMORY reaches 90.00% with GPT-4.1-Mini, below MemoryOS at 100.00%, and 70.00% with GPT-4o-Mini, below MemOS at 96.67%. This suggests that dedicated persona or user-profile modeling may provide advantages for preference-heavy queries. A future system could combine segment-level evidence construction with stronger profile modeling.

The experiments use hosted LLM and embedding APIs together with a reranker model or service. Replacing these components with compatible open-source models is possible, but model capability and token accounting may change; any such replacement should therefore be evaluated with newly reported accuracy and cost numbers.

Artifact release should separate code, prompts, configurations, question-id lists, prediction files, and token-accounting summaries from benchmark redistribution. The local LoCoMo license file specifies Creative Commons Attribution-NonCommercial 4.0 International (CC BY-NC 4.0), so artifacts containing LoCoMo source conversations, QA pairs, evidence, or substantial derived content should retain attribution and license notices and should respect the non-commercial restriction. The local LongMemEval reference copy is MIT licensed, which requires retaining copyright and license notices when redistributing code, evaluation scripts, processed identifiers, or derived metadata. If multiple benchmark resources are packaged together, each dataset should keep its upstream license instead of being relicensed under the code license of this paper.